

PROBLEMA 4

Paradigma Funcional Declarativo

Dataset: BMW Stock Data (12_bmw_data.csv)

7.212 registros bursátiles • 1996–2024

1. Introducción y contexto

Este documento constituye la documentación técnica completa del **Problema 4** del trabajo de Python de la asignatura Fundamentos de Sistemas de Información. El problema exige implementar un motor de procesamiento de datos bajo un **paradigma declarativo estricto**, donde el estado de los datos es inmutable y queda terminantemente prohibido el uso de estructuras iterativas (bucles `for` / `while`) o variables de estado temporales fuera de la cadena de procesamiento.

El dataset asignado es **12_bmw_data.csv**, que contiene 7.212 registros de cotización bursátil de BMW AG desde noviembre de 1996 hasta diciembre de 2024. Cada registro incluye la fecha de sesión, los precios de apertura, cierre, máximo, mínimo, cierre ajustado y el volumen de negociación.

1.1. Restricciones del enunciado

El enunciado impone cinco restricciones fundamentales que condicionan toda la arquitectura de la solución:

1. **Prohibición de bucles:** no se permite ninguna estructura iterativa (`for`, `while`) ni variables de estado temporales fuera de la cadena funcional. La solución debe ser una expresión única y continua.
2. **Funciones de orden superior:** el motor de transformación debe basarse en la composición de `filter`, `map` y `reduce`, donde el resultado de una operación alimenta a la siguiente.
3. **Fábricas de predicados:** se requieren funciones que generen otras funciones (closures) para aplicar criterios de filtrado dinámicos sin alterar el núcleo del sistema.
4. **Reducción compleja (plegado):** el cierre del pipeline debe reconstruir una estructura jerárquica y enriquecida a partir de los datos procesados, no una simple suma de valores.
5. **Comprensiones avanzadas multinivel:** se exigen comprensiones que operen en múltiples niveles de profundidad, realizando transformaciones cruzadas entre diferentes atributos en una sola línea de ejecución.

1.2. Estructura del dataset

El fichero CSV contiene las siguientes columnas con sus tipos de dato correspondientes:

Columna	Tipo	Prefijo (Notación Húngara)	Descripción
Date	str	sDate	Fecha de la sesión bursátil (YYYY-MM-DD)
Adj_Close	float	fAdjClose	Precio de cierre ajustado por dividendos y splits
Close	float	fClose	Precio de cierre de la sesión
High	float	fHigh	Precio máximo alcanzado en la sesión
Low	float	fLow	Precio mínimo alcanzado en la sesión

Columna	Tipo	Prefijo (Notación Húngara)	Descripción
Open	float	fOpen	Precio de apertura de la sesión
Volume	int	iVolume	Volumen de acciones negociadas

El rango de precios de cierre abarca desde **17,04 EUR** hasta **122,60 EUR**, cubriendo casi tres décadas de historia bursátil del fabricante alemán.

2. Arquitectura de la solución

El programa se organiza en **seis secciones funcionales** dentro de un único fichero **P4.py**. Cada sección está claramente delimitada por comentarios de bloque y responde a un requisito específico del enunciado. La siguiente tabla muestra la correspondencia entre secciones y requisitos:

Sección	Nombre	Requisito que cubre
1	Carga inmutable del dataset	Datos inmutables, sin bucles
2	Fábricas de predicados	Funciones que generan funciones
3	Pipeline filter → map → reduce	Cadena de funciones de orden superior
4	Comprensiones avanzadas multinivel	Transformaciones cruzadas en una línea
5	Funciones de presentación	Visualización sin bucles (map + join)
6	Programa principal	Demostración y validación

2.1. Dependencias e imports

El programa utiliza exclusivamente módulos de la biblioteca estándar de Python, sin dependencias externas:

```
import csv                # Lectura del fichero CSV
from functools import reduce # Función de plegado (reduce)
from datetime import datetime # (disponible para extensiones)
```

La elección de **csv** en lugar de Pandas es deliberada: el enunciado prohíbe variables de estado intermedias y exige expresiones puras, lo que hace que Pandas (orientado a mutación in-place) no encaje con el paradigma funcional.

3. Sección 1: Carga inmutable del dataset

La función `cargar_dataset(sRuta)` lee el fichero CSV y devuelve una **tupla de diccionarios**. Se elige `tuple` en lugar de `list` porque las tuplas son inmutables en Python, lo que garantiza que ningún código posterior pueda modificar los datos originales.

3.1. Flujo de la carga

1. **Lectura del CSV:** se abre el fichero con `csv.reader`, se extrae la cabecera con `next()` y se almacenan las filas restantes.
2. **Función de transformación:** se define `_transformar_fila(lFila)` que convierte una lista de strings crudos en un diccionario tipado. Cada campo se castea a su tipo correcto (`float` para precios, `int` para volumen) y se redondea a 4 decimales.
3. **Aplicación con `map()`:** en lugar de un bucle `for`, se usa `map(_transformar_fila, lFilas)` para aplicar la transformación a cada fila. El resultado se envuelve en `tuple()` para hacerlo inmutable.

3.2. Código comentado

```
def cargar_dataset(sRuta):
    with open(sRuta, newline='', encoding='utf-8') as oFichero:
        oLector = csv.reader(oFichero)
        lCabecera = next(oLector) # Extraemos la cabecera
        lFilas = list(oLector)    # Leemos todas las filas

    def _transformar_fila(lFila):
        return {
            "sDate":      lFila[0],          # str: fecha
            "fAdjClose":  round(float(lFila[1]), 4), # float
            "fClose":     round(float(lFila[2]), 4),
            "fHigh":      round(float(lFila[3]), 4),
            "fLow":       round(float(lFila[4]), 4),
            "fOpen":      round(float(lFila[5]), 4),
            "iVolume":    int(lFila[6])      # int: volumen
        }

    # map() aplica la transformación sin bucle
    return tuple(map(_transformar_fila, lFilas))
```

3.3. Notación húngara

Siguiendo el estilo del proyecto base (clases `Estudiante`, `Asignatura`, etc.), cada clave del diccionario lleva un prefijo de tipo: `s` para string, `f` para float, `i` para int. Esto facilita la lectura y evita errores de tipo en la cadena funcional.

4. Sección 2: Fábricas de predicados

Una fábrica de predicados es una **función que devuelve otra función**. Este patrón (conocido como *closure* en programación funcional) permite crear filtros dinámicos en tiempo de ejecución sin modificar el código fuente. Los parámetros de la función externa quedan «capturados» en el entorno de la función interna.

4.1. Catálogo de fábricas implementadas

Fábrica	Parámetros	Predicado generado	Uso típico
fabrica_filtro_rango	sCampo, fMinimo, fMaximo	fMinimo <= d[sCampo] <= fMaximo	Filtrar por rango de precios
fabrica_filtro_anio	iAnio	sDate empieza por str(iAnio)	Seleccionar un año concreto
fabrica_filtro_volumen_minimo	iUmbral	iVolume >= iUmbral	Descartar días de baja liquidez
fabrica_filtro_volatilidad_minima	fUmbral	(fHigh - fLow) >= fUmbral	Seleccionar días volátiles

4.2. Ejemplo detallado: fabrica_filtro_rango

```
def fabrica_filtro_rango(sCampo, fMinimo, fMaximo):
    # Devuelve una lambda que "recuerda" sCampo, fMinimo y fMaximo
    # gracias al mecanismo de closure de Python
    return lambda dRegistro: fMinimo <= dRegistro[sCampo] <= fMaximo

# Uso: crear un filtro para Close entre 90 y 120 EUR
fnFiltro = fabrica_filtro_rango("fClose", 90.0, 120.0)

# Aplicar el filtro con filter() (sin bucle)
tResultado = tuple(filter(fnFiltro, tDataset))
```

La ventaja clave es que `fnFiltro` es una función autónoma que se puede pasar como argumento a `filter()`, componer con otros predicados mediante `lambda`, o almacenar para reutilizarla más tarde. El núcleo del sistema no necesita saber qué campo ni qué rango se está filtrando.

4.3. Composición de predicados

Las fábricas se pueden combinar dinámicamente usando operadores lógicos dentro de una lambda envolvente:

```
fnFiltro2020 = fabrica_filtro_anio(2020)
fnFiltroVol = fabrica_filtro_volumen_minimo(5_000_000)

# Composición: ambas condiciones deben cumplirse
tResultado = tuple(filter(
    lambda d: fnFiltro2020(d) and fnFiltroVol(d),
```

```
tDataset  
)
```

5. Sección 3: Pipeline funcional (filter → map → reduce)

La función `pipeline_analisis_anual(tDataset, iAnio, fVolatilidadMin)` implementa la cadena completa de procesamiento funcional en tres fases secuenciales. Esta es la pieza central del Problema 4 y la que demuestra la composición de funciones de orden superior.

5.1. Fase 1: Segmentación (filter)

Se combinan dos fábricas de predicados (`fabrica_filtro_anio` + `fabrica_filtro_volatilidad_minima`) en un solo filtro compuesto. La función `filter()` recorre el dataset completo y selecciona únicamente los registros que cumplen ambas condiciones simultáneamente.

```
fnFiltroAnio = fabrica_filtro_anio(iAnio)
fnFiltroVolatilidad = fabrica_filtro_volatilidad_minima(fVolatilidadMin)

tFiltrado = tuple(filter(
    lambda dReg: fnFiltroAnio(dReg) and fnFiltroVolatilidad(dReg),
    tDataset
))
```

Por ejemplo, para el año 2020 con volatilidad mínima de 1,0 EUR, de los 7.212 registros totales se seleccionan 229 días que cumplen ambos criterios.

5.2. Fase 2: Transformación (map)

Cada registro filtrado pasa por `map()`, que lo enriquece con tres campos derivados calculados al vuelo sin mutar el registro original (se usa el operador spread `**dReg` para crear un nuevo diccionario):

Campo derivado	Fórmula	Significado
fVolatilidad	fHigh - fLow	Rango intradía en EUR
fRendimiento	((fClose - fOpen) / fOpen) × 100	Variación porcentual apertura-cierre
sMes	sDate[5:7]	Mes extraído de la fecha (01-12)

5.3. Fase 3: Consolidación (reduce)

Esta es la fase más compleja. La función `reduce()` de `functools` recorre toda la secuencia transformada y va construyendo un **diccionario jerárquico acumulador**. A diferencia de una simple suma, el acumulador contiene:

- Métricas globales: contador de registros, sumas de cierre/volatilidad/rendimiento, máximo histórico, mínimo histórico, volumen total y fecha del día más volátil.
- Sub-diccionario jerárquico dMeses: un diccionario anidado donde cada clave es un mes ("01".."12") y cada valor contiene el recuento de días, suma de cierres y suma de volatilidades de ese mes.

El valor semilla (estado inicial del acumulador) es un diccionario con todos los campos inicializados a cero o vacíos. Con cada registro procesado, **reduce** genera un nuevo diccionario (inmutabilidad) que incorpora la información del registro actual. El sub-diccionario **dMeses** se actualiza mediante spread ****dAcc["dMeses"]** y **.get()** con valores por defecto, evitando cualquier mutación.

5.4. Post-procesado funcional

Tras el reduce, se calculan las medias dividiendo las sumas entre el número de registros. El desglose mensual se enriquece con **fMediaClose** y **fMediaVolatilidad** usando una **comprensión de diccionario** (sin bucle for imperativo).

5.5. Estructura del reporte generado

El diccionario resultante tiene esta estructura jerárquica:

```
{
  "iAnio": 2020,
  "iRegistros": 229,
  "fMediaClose": 59.92,
  "fMaxHigh": 77.31,
  "fMinLow": 36.59,
  "fMediaVolatilidad": 1.883,
  "fMediaRendimiento": -0.0101,
  "iSumaVolumen": 490302390,
  "sFechaMasVolatil": "2020-03-13",
  "dMeses": {
    "01": { "iDias": 16, "fMediaClose": 70.21, "fMediaVolatilidad": 1.43 },
    "02": { "iDias": 16, "fMediaClose": 63.85, "fMediaVolatilidad": 1.68 },
    "03": { "iDias": 22, "fMediaClose": 47.61, "fMediaVolatilidad": 2.98 },
    ...
  }
}
```

6. Sección 4: Comprensiones avanzadas multinivel

Esta sección demuestra el uso de comprensiones de lista y diccionario que operan en múltiples niveles de profundidad, cruzando atributos independientes en una sola expresión. Se implementan tres comprensiones distintas:

6.1. Clasificación cruzada: `comprehension_clasificacion_cruzada`

Genera una lista de tuplas donde cada registro se clasifica simultáneamente en dos dimensiones independientes (volatilidad intradía y dirección del mercado), combinando criterios de múltiples atributos en una sola expresión:

```
[
    (
        dReg["sDate"],
        "ALTA_VOLATILIDAD" if (dReg["fHigh"] - dReg["fLow"]) > 2.0
        else "BAJA_VOLATILIDAD",
        "ALCISTA" if dReg["fClose"] > dReg["fOpen"]
        else ("BAJISTA" if dReg["fClose"] < dReg["fOpen"]
        else "NEUTRO"),
        dReg["fClose"],
        dReg["iVolume"]
    )
    for dReg in tDataset
    if dReg["iVolume"] > 0
]
```

El umbral de 2,0 EUR para clasificar como alta volatilidad se basa en que el rango intradía medio histórico de BMW ronda los 1,5 EUR. Así, un día con rango superior a 2 EUR representa una volatilidad significativamente por encima de la media.

6.2. Matriz década x mes: `comprehension_resumen_decada_mes`

Esta es la comprensión más compleja del programa. Genera un **diccionario de diccionarios** con tres niveles de profundidad anidada en una sola expresión:

1. **Nivel exterior:** itera sobre las décadas únicas (1990s, 2000s, 2010s, 2020s), extraídas con una set comprehension.
2. **Nivel medio:** para cada década, itera sobre los 12 meses.
3. **Nivel interior:** cuenta los registros que coinciden con esa década Y ese mes usando una list comprehension con filtro doble.

```
{
    sDecada: {
        sMes: len([
            dReg for dReg in tDataset
            if dReg["sDate"][:3] + "0s" == sDecada
            and dReg["sDate"][5:7] == sMes
        ])
        for sMes in sorted(setMeses)
    }
}
```

```
    for sDecada in sorted(setDecadas)
  }
```

6.3. Top N días extremos: `comprehension_top_dias_extremos`

Combina una list comprehension con `sorted()` y slicing funcional `[ :iTopN]` para identificar los N días con mayor rango intradía, enriqueciendo cada resultado con la dirección del mercado y el rendimiento porcentual. Todo sin variables auxiliares ni bucles.

7. Sección 5: Funciones de presentación

Incluso la presentación por consola sigue el paradigma funcional. En lugar de bucles `for` para recorrer las filas, se usa el patrón **map + join**: se transforma cada elemento en su representación textual con `map()` y se concatenan todas las líneas con `"\n".join()`.

Ejemplo del patrón aplicado al desglose mensual:

```
lLines = list(map(
    lambda tItem: f" {tItem[0]:<6} {tItem[1]['iDias']:>6} ...",
    sorted(dReporte["dMeses"].items())
))
print("\n".join(lLines))
```

Se implementan cuatro funciones de presentación: `mostrar_reporte_anual` para el pipeline, `mostrar_clasificacion_cruzada` para la clasificación, `mostrar_top_extremos` para el ranking, y `mostrar_decada_mes` para la matriz. Todas siguen el mismo patrón map + join.

8. Sección 6: Programa principal y demostración

La función `main()` orquesta tres demostraciones que ponen a prueba todas las capacidades del sistema:

8.1. Demo 1: Fábricas de predicados

Se crean dos filtros dinámicos y se aplican al dataset completo: uno para precios de cierre entre 90-120 EUR (resultado: 772 días) y otro combinando año 2020 + volumen superior a 5 millones (resultado: 6 días). Se muestran los primeros resultados en tabla.

8.2. Demo 2: Pipeline funcional completo

Se ejecuta `pipeline_analisis_anual` para dos años: 2020 (volatilidad mín. 1,0 EUR) y 2015 (volatilidad mín. 0,5 EUR). El reporte incluye métricas globales y desglose mensual. Destaca que el día más volátil de 2020 fue el 13 de marzo (inicio del crash del COVID-19) y el de 2015 el 24 de septiembre (escándalo de emisiones Dieselgate de Volkswagen, que arrastró a BMW).

8.3. Demo 3: Comprensiones avanzadas

Se ejecutan las tres comprensiones multinivel: la clasificación cruzada revela que hay 602 días alcistas de alta volatilidad frente a 632 bajistas (ligero sesgo negativo en días turbulentos). El top 10 muestra que los días más volátiles son predominantemente bajistas (8 de 10). La matriz década/mes confirma la cobertura temporal completa del dataset.

9. Verificación del cumplimiento

Para demostrar que el programa cumple estrictamente con las restricciones del enunciado, se realiza un análisis exhaustivo del código:

9.1. Ausencia de bucles imperativos

Se ha verificado con `grep` que no existe ninguna sentencia `for` ni `while` como estructura iterativa independiente. Todas las apariciones de la palabra `for` en el código están **dentro de comprensiones** (list comprehension o dict comprehension), que son expresiones declarativas, no bucles imperativos. La diferencia es fundamental:

Bucle imperativo (PROHIBIDO)	Comprensión declarativa (PERMITIDO)
for x in lista: acum += x	[x for x in lista if condicion]
Ejecuta sentencias paso a paso	Evalúa una expresión completa
Requiere variables de estado	Sin efectos secundarios
Modifica estado externo	Devuelve un valor nuevo

9.2. Tabla de correspondencia requisitos → código

Requisito del enunciado	Elemento del código	Línea(s)
Sin bucles ni variables temporales	map(), filter(), reduce(), comprensiones	Todo el fichero
Cadena filter → map → reduce	pipeline_analisis_anual()	101–173
Fábricas de predicados (closures)	fabrica_filtro_rango, _anio, _volumen, _volatilidad	58–92
Plegado (reduce) complejo jerárquico	reduce() con acumulador dict + dMeses anidado	142–173
Comprensiones avanzadas multinivel	3 funciones de comprensión cruzada	205–265
Inmutabilidad de datos	tuple() para dataset, spread ** para dicts	49, 131

10. Conclusiones

La implementación del Problema 4 demuestra que es posible construir un motor de análisis de datos completo bajo un paradigma puramente funcional. El uso de `filter`, `map` y `reduce` como bloques fundamentales, combinado con fábricas de predicados y comprensiones multinivel, permite procesar 7.212 registros bursátiles generando reportes jerárquicos con desglose mensual, clasificaciones cruzadas y rankings, todo ello sin una sola línea de código imperativo.

El enfoque funcional aporta tres ventajas principales: la inmutabilidad previene errores por efectos secundarios, la composición de funciones permite construir pipelines complejos a partir de piezas simples, y las fábricas de predicados hacen que el sistema sea extensible sin modificar su núcleo.